

6 ПРАКТИЧЕСКАЯ РАБОТА (ФИЗИЧЕСКОЕ ПРОЕКТИРОВАНИЕ И СОЗДАНИЕ БАЗЫ ДАННЫХ)

6.1 Цель работы

Целью практической работы является формирование навыков преобразования разработанной логической модели данных в физическую структуру реляционной базы данных средствами выбранной СУБД, включая определение таблиц, типов данных, первичных и внешних ключей, ограничений целостности и правил ссылочной целостности.

6.2 Задачи работы

В процессе выполнения практической работы необходимо:

- проанализировать утверждённую логическую модель данных;
- выбрать реляционную систему управления базами данных для реализации проекта;
- преобразовать сущности логической модели в таблицы;
- определить физические имена таблиц и столбцов;
- определить типы данных столбцов;
- определить первичные ключи;
- реализовать внешние ключи;
- определить обязательность хранения значений;
- реализовать ограничения уникальности;

- реализовать проверки допустимости значений;
- определить правила ссылочной целостности при изменении и удалении связанных записей;
- разработать SQL-скрипт создания базы данных;
- обеспечить воспроизводимость создания физической структуры;
- проверить соответствие физической реализации утверждённой логической модели;
- подготовить базу данных к последующему наполнению и разработке SQL-запросов.

6.3 Теоретические сведения

Структурированное хранение данных предполагает размещение информации в заранее определённых форматах и схемах.

К характерным особенностям структурированного хранения относятся:

- наличие фиксированной схемы;
- определённость форматов хранения;
- взаимосвязанность данных;
- возможность выполнения формализованных запросов;
- возможность контроля качества и целостности данных.

Реляционная база данных является одним из наиболее распространённых способов реализации структурированного хранения.

В реляционной модели данные организуются в виде взаимосвязанных отношений, которые при физической реализации в реляционной СУБД представляются таблицами.

Переход от логической модели к физической реализации предполагает принятие технических решений, связанных с особенностями выбранной СУБД.

На данном этапе необходимо определить:

- физическую структуру таблиц;
- типы данных;
- ключевые ограничения;
- ограничения допустимых значений;
- правила ссылочной целостности;
- используемые средства СУБД для обеспечения корректности данных.

6.3.1 Физическая модель базы данных

Физическая модель представляет способ непосредственной реализации логической структуры данных средствами конкретной системы управления базами данных.

Если логическая модель отвечает на вопрос:

какие данные и отношения необходимо представить,

то физическая модель определяет:

каким образом эти данные будут реализованы в выбранной СУБД.

При переходе к физической модели необходимо установить соответствие:

- сущность → таблица;
- атрибут → столбец;
- идентификатор → первичный ключ;
- отношение между сущностями → внешний ключ или иная физическая конструкция;

- бизнес-ограничение → ограничение целостности, серверная логика или программная проверка.

При этом указанное соответствие не всегда является полностью механическим.

Особенности реализации могут потребовать:

- уточнения типов данных;
- введения технических идентификаторов;
- создания дополнительных ограничений;
- изменения физических наименований объектов;
- уточнения правил удаления и изменения связанных записей.

Физическая реализация не должна изменять смысл утверждённой логической модели без соответствующего изменения самой логической модели.

6.3.2 Выбор СУБД

Для выполнения проекта должна быть выбрана реляционная СУБД.

Выбор СУБД должен обеспечивать возможность последующей реализации:

- SQL-запросов;
- ограничений целостности;
- функций и/или хранимых процедур;
- триггеров;
- пользователей и ролей;
- подключения разрабатываемого приложения.

При выборе рекомендуется учитывать:

- доступность СУБД;

- наличие необходимых средств разработки;
- поддержку требуемых типов данных;
- поддержку серверной логики;
- возможность подключения из выбранного языка программирования;
- наличие средств администрирования и диагностики.

После начала реализации проекта смена СУБД может потребовать переработки SQL-скриптов, серверной логики и программного кода.

Поэтому выбор СУБД должен быть выполнен до начала разработки окончательной физической схемы.

6.3.3 Таблицы

Каждая основная сущность логической модели, предназначенная для постоянного хранения данных, должна быть физически представлена средствами выбранной СУБД.

В большинстве случаев для этого используется таблица.

Наименование таблицы должно:

- однозначно отражать её назначение;
- соответствовать принятому в проекте стилю именования;
- не содержать случайных сокращений;
- быть согласовано с остальными объектами базы данных.

В рамках одного проекта рекомендуется применять единый подход к именованию.

Например:

- users
- orders

- order_items
- payments

либо иной единый вариант.

Не рекомендуется одновременно использовать несогласованные обозначения:

- Users
- tbl_order
- PaymentInfo
- список_товаров

если это не обусловлено специальными требованиями проекта.

6.3.4 Типы данных

Для каждого столбца необходимо выбрать тип данных, соответствующий смыслу хранимой информации.

Выбор типа должен учитывать:

- характер значения;
- допустимый диапазон;
- необходимость выполнения вычислений;
- требуемую точность;
- возможность сортировки и сравнения;
- особенности выбранной СУБД.

Не рекомендуется использовать строковый тип для всех без исключения данных.

Например:

- дата должна храниться с использованием типа даты или даты и времени;
- целочисленное количество — с использованием целочисленного типа;
- денежные и иные точные величины — с использованием подходящего точного числового типа;
- логические значения — с использованием логического типа, если он поддерживается СУБД.

Выбор конкретного типа должен соответствовать семантике атрибута.

Например, телефонный номер обычно не является числом в математическом смысле, поскольку над ним не выполняются арифметические операции и он может содержать дополнительные символы.

Следовательно, его хранение в числовом поле может быть некорректным проектным решением.

6.3.5 Первичный ключ

Первичный ключ предназначен для однозначной идентификации строки таблицы.

Каждая основная таблица должна иметь первичный ключ.

При использовании суррогатного идентификатора он может быть реализован как отдельный технический столбец, например:

- user_id
- order_id
- payment_id

Наличие суррогатного первичного ключа не отменяет необходимости реализации ограничений уникальности для значений, которые должны быть уникальными согласно правилам предметной области.

Например, наличие `user_id` не означает, что в системе автоматически допустимо создание нескольких пользователей с одним и тем же уникальным логином.

6.3.6 Внешние ключи

Внешний ключ используется для реализации ссылочной связи между таблицами.

Внешний ключ должен ссылаться на:

- первичный ключ;
- либо иной кандидатный ключ, имеющий ограничение уникальности.

Использование внешних ключей обеспечивает сохранение ссылочной целостности данных.

Например, если таблица заказов содержит идентификатор клиента, СУБД должна предотвращать появление заказа, ссылающегося на несуществующего клиента.

Не допускается замена внешнего ключа обычным числовым или строковым столбцом без ограничения `FOREIGN KEY`, если между таблицами существует обязательная ссылочная связь.

6.3.7 Обязательность значений

Ограничение NOT NULL используется в случаях, когда отсутствие значения недопустимо по правилам предметной области.

Например, для заказа обязательными могут являться:

- идентификатор клиента;
- дата создания;
- состояние.

В то же время некоторые сведения могут быть необязательными.

Решение о допустимости NULL должно приниматься на основании предметной области.

Не рекомендуется:

- устанавливать NOT NULL для всех столбцов без анализа;
- разрешать NULL для всех столбцов исключительно для упрощения заполнения базы.

6.3.8 Ограничение уникальности

Ограничение UNIQUE используется для значений или комбинаций значений, которые не должны повторяться.

Примерами могут являться:

- логин пользователя;
- номер договора;
- артикул товара;
- номер документа;
- комбинация значений, уникальная в рамках конкретного контекста.

Ограничение уникальности должно быть связано с реальным правилом предметной области.

Не следует создавать UNIQUE исключительно ради выполнения минимального количественного требования.

6.3.9 Ограничение CHECK

Ограничение CHECK предназначено для проверки допустимости значения или комбинации значений непосредственно средствами СУБД.

Оно может применяться для реализации правил вида:

- количество должно быть положительным;
- сумма не может быть отрицательной;
- дата окончания не должна предшествовать дате начала;
- значение должно находиться в установленном диапазоне.

Пример логического условия:

`quantity > 0`

или:

`end_date >= start_date`

Ограничение должно отражать содержательное правило предметной области.

Проверка вида:

`id > 0`

при автоматически формируемом положительном идентификаторе обычно не рассматривается как содержательное ограничение предметной области.

6.3.10 Ссылочная целостность

При наличии связанных таблиц необходимо определить поведение системы при изменении или удалении родительской записи.

В зависимости от предметной области могут применяться различные стратегии:

- запрет удаления связанной записи;
- каскадное удаление зависимых данных;
- установка внешнего ключа в NULL;
- каскадное изменение идентификатора;
- иные механизмы, поддерживаемые выбранной СУБД.

Выбор стратегии должен быть обоснован.

Каскадное удаление не должно применяться автоматически ко всем связям.

Например, удаление клиента не всегда должно приводить к удалению всей истории его заказов.

Для исторически значимых данных более корректным решением может быть запрет физического удаления либо использование иной модели жизненного цикла объекта.

6.3.11 Физическая реализация бизнес-правил

Не каждое бизнес-правило должно реализовываться одним и тем же способом.

В зависимости от характера правила могут применяться:

- NOT NULL;
- UNIQUE;

- СHECK;
- первичные и внешние ключи;
- функции;
- процедуры;
- триггеры;
- программная логика приложения.

На этапе настоящей практической работы необходимо реализовать те правила, которые естественным образом выражаются декларативными ограничениями структуры базы данных.

Более сложная серверная логика будет разработана в последующих практических работах.

Например:

«сумма платежа не может быть отрицательной»

целесообразно реализовать средствами ограничения данных.

Правило:

«после подтверждения оплаты необходимо изменить состояние заказа и создать запись в журнале»

требует более сложной логики и может быть реализовано на последующем этапе.

6.3.12 SQL-скрипт создания базы данных

Физическая структура должна создаваться посредством SQL-скрипта.

Основным результатом настоящей работы является файл:

`01_create_tables.sql`

Скрипт должен обеспечивать последовательное создание необходимых объектов базы данных.

При его подготовке следует учитывать порядок зависимостей.

Таблица, на которую ссылается внешний ключ, должна существовать к моменту создания соответствующего ограничения либо ограничение должно быть добавлено позднее.

SQL-скрипт должен быть:

- читаемым;
- структурированным;
- воспроизводимым;
- пригодным для повторного использования.

Рекомендуется использовать комментарии для обозначения основных разделов.

6.3.13 Воспроизводимость базы данных

Одним из обязательных требований настоящей практической работы является возможность создать базу данных на основании подготовленного SQL-скрипта.

Недостаточно создать таблицы вручную через графический интерфейс СУБД и продемонстрировать существующую базу.

Проект должен содержать программно воспроизводимое описание структуры.

Проверка может выполняться следующим образом:

- создаётся новая пустая база данных;
- выполняется файл `01_create_tables.sql`;
- скрипт завершается без необработанных ошибок;
- в базе появляются все предусмотренные таблицы и ограничения;

- полученная структура соответствует физической модели проекта.

Внесение изменения непосредственно через GUI СУБД без отражения этого изменения в SQL-скрипте считается нарушением требования воспроизводимости.

6.4 Задание

На основании утверждённой в ПР №5 логической модели необходимо реализовать физическую структуру реляционной базы данных.

6.4.1 Выбор СУБД

Необходимо указать:

- выбранную СУБД;
- используемую версию, если это существенно;
- краткое обоснование выбора.

6.4.2 Преобразование сущностей в таблицы

Для каждой сущности необходимо определить:

- имя таблицы;
- набор столбцов;
- первичный ключ;

- обязательные и необязательные значения;
- ограничения уникальности;
- внешние ключи;
- дополнительные ограничения.

6.4.3 Определение типов данных

Для каждого столбца необходимо определить физический тип данных выбранной СУБД.

Выбор типов должен быть обоснован семантикой данных.

6.4.4 Реализация ключей

Необходимо реализовать:

- первичные ключи;
- внешние ключи;
- при наличии — составные ключи;
- ограничения уникальности естественных идентификаторов.

6.4.5 Реализация ограничений

Необходимо реализовать предусмотренные моделью ограничения:

- NOT NULL;

- UNIQUE;
- CHECK;
- ограничения ссылочной целостности.

6.4.6 Разработка SQL-скрипта

Необходимо сформировать единый SQL-скрипт создания физической структуры базы данных.

Скрипт должен быть сохранён в структуре проекта:

```
/sql/01_create_tables.sql
```

6.4.7 Проверка развёртывания

После подготовки скрипта необходимо:

- удалить или создать заново тестовую базу;
- выполнить скрипт;
- проверить отсутствие ошибок;
- проверить структуру таблиц;
- проверить наличие ключей и ограничений;
- зафиксировать результат проверки.

6.5 Минимальные требования к результату

Для зачёта практической работы физическая база данных должна содержать:

- от 6 до 8 и более содержательно обоснованных связанных таблиц в соответствии с утверждённой моделью;
- первичный ключ для каждой основной таблицы;
- не менее 3 внешних ключей;
- корректно выбранные типы данных;
- использование NOT NULL для обязательных данных;
- не менее 2 содержательных ограничений CHECK;
- не менее 2 содержательных ограничений UNIQUE;
- определённые правила ссылочной целостности;
- полностью воспроизводимый SQL-скрипт создания структуры.

Если утверждённая в ПР №5 модель содержит более восьми сущностей, уменьшать физическую структуру до восьми таблиц не требуется.

Количество 6–8 является минимальным ориентиром, а не верхним ограничением.

Не допускается искусственное создание технически бессмысленных таблиц, ограничений или связей исключительно для достижения количественных требований.

6.5.1 Требования к соответствию логической модели

Физическая структура должна соответствовать утверждённой логической модели.

Необходимо проверить:

- каждая сущность, предназначенная для хранения, получила физическую реализацию;
- основные атрибуты реализованы столбцами;
- идентификаторы реализованы ключами;
- связи реализованы внешними ключами;
- кардинальности не искажены;
- ассоциативные сущности корректно реализованы;
- обязательность отношений отражена соответствующими ограничениями;
- естественные требования уникальности не потеряны.

Если при физическом проектировании выявлена ошибка логической модели, необходимо сначала актуализировать ПР №5, а затем реализовать исправленную структуру.

Не допускается ситуация, при которой ER-модель и фактическая база данных описывают различные структуры.

6.5.2 Проверка ограничений

Необходимо выполнить первичную проверку реализованных ограничений.

Для каждого содержательного ограничения следует проверить:

- допустимый сценарий;
- сценарий нарушения ограничения.

Например, для:

ЧЕКС (quantity > 0)

следует убедиться, что:

- положительное количество принимается;
- нулевое или отрицательное значение отклоняется.

Для внешнего ключа следует убедиться, что:

- ссылка на существующий объект допускается;
- ссылка на несуществующий объект отклоняется.

Подробная система тест-кейсов будет разработана позднее, однако уже на настоящем этапе необходимо подтвердить корректность физической схемы.

6.6 Порядок выполнения работы

- Проанализировать замечания по ПР №5.
- Убедиться в прохождении контрольной точки №2.
- Выбрать реляционную СУБД.
- Зафиксировать правила именования объектов.
- Преобразовать сущности в таблицы.
- Определить физические типы данных.
- Реализовать первичные ключи.
- Реализовать внешние ключи.
- Реализовать обязательность значений.
- Реализовать естественную уникальность данных.
- Реализовать ограничения СHECK.
- Определить правила ссылочной целостности.
- Сформировать `01_create_tables.sql`.
- Выполнить скрипт на пустой базе.

- Исправить выявленные ошибки.
- Проверить структуру созданной базы.
- Сопоставить структуру с логической моделью.
- Проверить несколько корректных и ошибочных операций.
- Подготовить отчёт.
- Подготовить SQL-скрипт к защите.

6.7 Содержание отчёта

Отчёт по практической работе должен содержать:

- титульный лист;
- номер и название практической работы;
- цель работы;
- сведения о выбранной СУБД;
- краткое обоснование выбора;
- принятую систему именования объектов;
- физическое описание таблиц;
- сведения о первичных ключах;
- сведения о внешних ключах;
- описание ограничений NOT NULL, UNIQUE и CHECK;
- описание правил ссылочной целостности;
- фрагменты SQL-кода создания основных объектов;
- результаты проверки выполнения скрипта;
- результаты проверки ограничений;

- сведения об изменениях логической модели, если они выполнялись;
- вывод по результатам работы.

Не требуется включать полный SQL-скрипт в основной текст отчёта, если его объём существенно снижает читаемость документа.

Полный скрипт должен быть представлен отдельным файлом.

6.8 Результаты, предоставляемые преподавателю

Для защиты практической работы обучающийся предоставляет:

- отчёт;
- файл `01_create_tables.sql`;
- актуальную логическую модель;
- развёрнутую тестовую базу данных либо возможность её создать;
- при необходимости — актуализированные материалы ПР №5.

Рекомендуемая структура проекта:

/project
/diagrams
er_model.png
/sql
01_create_tables.sql

При защите обучающийся должен быть готов:

- создать базу данных из SQL-скрипта;
- объяснить назначение любой таблицы;
- обосновать выбор типов данных;
- объяснить любой первичный и внешний ключ;
- объяснить назначение ограничений;

- продемонстрировать реакцию СУБД на нарушение ограничения;
- обосновать правила удаления связанных данных;
- показать соответствие физической структуры ER-модели.

6.9 Использование результатов в курсовой работе

Физическая структура базы данных является одним из основных технических результатов курсовой работы.

Для зачёта настоящей практической работы достаточно выполнения установленных минимальных требований.

В курсовой работе физическая модель должна полностью соответствовать окончательной версии системы.

Это означает необходимость обеспечить:

- реализацию всех сущностей итоговой логической модели;
- корректные физические типы данных;
- полный набор необходимых ключей;
- ограничения целостности;
- естественные ограничения уникальности;
- обоснованные правила ссылочной целостности;
- воспроизводимость создания базы данных.

Минимальные значения:

- 6–8 таблиц;
- 3 внешних ключа;
- 2 CHECK;
- 2 UNIQUE

не следует рассматривать как достаточный объём для курсовой работы, если фактическая предметная область требует более сложной структуры.

В курсовой работе количество ограничений должно определяться бизнес-правилами системы.

Кроме того, в окончательной физической модели могут потребоваться дополнительные средства организации и упорядочивания данных, в том числе индексы и иные объекты СУБД, если необходимость их применения обоснована характером запросов и условиями эксплуатации системы.

Следует учитывать, что после настоящей практической работы на структуру БД будут опираться:

- тестовые данные;
- SQL-запросы;
- функции;
- процедуры;
- триггеры;
- модель доступа;
- программный интерфейс;
- тестовые сценарии.

Поэтому существенное изменение структуры на поздних этапах может вызвать необходимость переработки значительной части проекта.

Обучающимся, использующим проект как основу курсовой работы, рекомендуется уже на ПР №6 реализовывать структуру, максимально близкую к предполагаемой итоговой базе данных.

6.10 Связь с контрольной точкой №2

Практическая работа №6 выполняется после прохождения контрольной точки №2 — фиксации базовой логической модели данных.

Если ПР №5 не защищена, окончательная физическая реализация считается преждевременной.

Допускается использование СУБД для экспериментальной проверки отдельных решений, однако итоговый 01_create_tables.sql должен основываться на согласованной логической модели.

После защиты ПР №6 любое изменение базовой структуры должно одновременно отражаться:

- в логической модели;
- в SQL-скрипте создания базы;
- во всех последующих зависимых материалах.

6.11 Вспомогательные материалы

Для преобразования логической модели рекомендуется использовать таблицу:

Элемент логической модели	Физическая реализация
Сущность	Таблица
Атрибут	Столбец
Идентификатор	PRIMARY KEY
Уникальный естественный признак	UNIQUE
Связь 1:M	FOREIGN KEY на стороне M
Ассоциативная сущность	Отдельная таблица
Обязательный атрибут	NOT NULL
Ограничение диапазона	CHECK
Ссылочная связь	FOREIGN KEY + правило удаления/изменения

Для каждой таблицы рекомендуется предварительно сформировать описание:

Столбец	Назначение	Тип	NULL	Ограничение
order_id	Идентификатор заказа	...	нет	PK
client_id	Клиент	...	нет	FK
status	Состояние заказа	...	нет	CHECK

При проверке каждого столбца рекомендуется ответить:

- соответствует ли тип реальному смыслу данных;
- может ли значение отсутствовать;
- должно ли значение быть уникальным;
- существуют ли ограничения диапазона;
- является ли столбец частью связи.

6.12 Типичные ошибки

К наиболее распространённым ошибкам относятся:

Ошибки структуры

- физическая база не соответствует ER-модели;
- отсутствие отдельных таблиц логической модели;
- появление новых таблиц без обновления модели;
- неправильная реализация ассоциативной сущности;
- сохранение связи M:N без промежуточной таблицы.

Ошибки типов данных

- использование строкового типа для всех значений;
- хранение дат как обычных строк;
- использование числового типа для телефонных номеров;

- недостаточная точность числовых значений;
- необоснованно большие размеры строковых полей.

Ошибки ключей

- отсутствие первичного ключа;
- использование изменяемого бизнес-значения как технического идентификатора без анализа;
- отсутствие внешнего ключа при существующей связи;
- внешний ключ не соответствует типу родительского ключа;
- использование суррогатного ключа при потере естественной уникальности.

Ошибки ограничений

- отсутствие NOT NULL для обязательных данных;
- создание формальных CHECK, не имеющих предметного смысла;
- отсутствие UNIQUE для естественно уникальных значений;
- реализация всех проверок исключительно в GUI;
- использование ограничений, противоречащих требованиям системы.

Ошибки ссылочной целостности

- каскадное удаление всех зависимых данных без анализа;
- отсутствие определённого поведения при удалении родительской записи;
- возможность появления ссылок на несуществующие объекты;
- потеря исторически значимых данных вследствие некорректного каскадного удаления.

Ошибки организации проекта

- создание таблиц вручную без SQL-скрипта;
- SQL-скрипт не соответствует фактической базе;

- необходимость вручную исправлять скрипт после его запуска;
- неправильный порядок создания зависимых объектов;
- отсутствие комментариев и структурирования в большом SQL-файле.

6.13 Контрольные вопросы

- В чём заключается различие между логической и физической моделью базы данных?
- Что понимается под структурированным хранением данных?
- Какие решения принимаются при физическом проектировании?
- По каким принципам выбирается тип данных столбца?
- Для чего используется первичный ключ?
- Для чего используется внешний ключ?
- Почему внешний ключ нельзя без основания заменить обычным числовым столбцом?
- Для чего применяется NOT NULL?
- Для чего применяется UNIQUE?
- Для чего применяется CHECK?
- В чём заключается ссылочная целостность?
- Какие варианты поведения могут использоваться при удалении родительской записи?
- Почему каскадное удаление не следует применять автоматически ко всем связям?
- Почему суррогатный первичный ключ не отменяет естественной уникальности?

- Какие бизнес-правила целесообразно реализовывать декларативными ограничениями СУБД?
- Почему база данных должна воспроизводиться из SQL-скрипта?
- Какие последствия может иметь изменение физической структуры после разработки запросов и серверной логики?
- Каким образом необходимо поддерживать согласованность физической базы данных и ER-модели?

6.14 Критерии приёмки работы

Практическая работа может быть зачтена при одновременном выполнении следующих условий:

- используется реляционная СУБД;
- структура соответствует утверждённой логической модели;
- реализовано не менее 6 содержательных связанных таблиц;
- все основные таблицы имеют первичные ключи;
- реализовано не менее 3 внешних ключей;
- типы данных выбраны корректно;
- обязательность данных отражена средствами NOT NULL;
- реализовано не менее 2 содержательных CHECK;
- реализовано не менее 2 содержательных UNIQUE;
- определены правила ссылочной целостности;
- отсутствуют необоснованные каскадные операции;
- SQL-скрипт создаёт структуру на пустой базе без необходимости ручного создания объектов;

- фактическая база соответствует 01_create_tables.sql;
- ограничения проверены на корректных и ошибочных данных;
- отсутствует искусственное увеличение количества объектов;
- обучающийся способен объяснить принятые физические решения.

6.15 Срок выполнения и сдачи

Срок выполнения и защиты практической работы определяется календарным графиком практических занятий.

Практическая работа №6 должна быть завершена до начала массового наполнения базы данных и разработки основного набора SQL-запросов.

Результатом настоящего этапа должна являться стабильная физическая схема, пригодная для использования во всех последующих практических работах.